



On planning sightseeing tours with TRIPBUILDER



Igo Ramalho Brilhante^a, Jose Antonio Macedo^a, Franco Maria Nardini^{b,*}, Raffaele Perego^b, Chiara Renso^b

^a Federal University of Ceará, Fortaleza, Brazil

^b ISTI-CNR, Pisa, Italy

ARTICLE INFO

Article history:

Received 12 May 2014

Received in revised form 3 October 2014

Accepted 8 October 2014

Available online 16 November 2014

Keywords:

Recommender systems

Trajectory mining

Sightseeing tours

ABSTRACT

We propose TRIPBUILDER, an unsupervised framework for planning personalized sightseeing tours in cities. We collect categorized Points of Interests (PoIs) from Wikipedia and albums of geo-referenced photos from Flickr. By considering the photos as traces revealing the behaviors of tourists during their sightseeing tours, we extract from photo albums spatio-temporal information about the itineraries made by tourists, and we match these itineraries to the Points of Interest (PoIs) of the city. The task of recommending a personalized sightseeing tour is modeled as an instance of the Generalized Maximum Coverage (GMC) problem, where a measure of personal interest for the user given her preferences and visiting time-budget is maximized. The set of actual trajectories resulting from the GMC solution is scheduled on the tourist's agenda by exploiting a particular instance of the Traveling Salesman Problem (TSP). Experimental results on three different cities show that our approach is effective, efficient and outperforms competitive baselines.

© 2014 Elsevier Ltd. All rights reserved.

1. Introduction

Tourists approaching their destination for the first time have to deal with the problem of planning a sightseeing itinerary that covers the most subjectively interesting attractions, and fits the time available for their visit. Precious information can be nowadays gathered from many digital sources, e.g., travel guides, maps, institutional sites, travel blogs. Nevertheless, the users still need to choose the preferred PoIs, guess how much time is needed to visit them and to move from one attraction to the next one. In this paper we discuss TRIPBUILDER, an unsupervised system helping tourists to build their own personalized sightseeing tour. Given the target destination, the time available for the visit, and the user's profile, our system recommends a time-budgeted tour that maximizes user's interests and takes into account both the time needed to enjoy the attractions and to move from one PoI to the next one. Moreover, the knowledge base feeding TRIPBUILDER recommendation model is entirely and automatically extracted from publicly available Web services, namely, Wikipedia, Flickr and Google maps.

We observe that an increasing number of tourists share on social networks their travel photos. Unofficial estimates state that Flickr, one of the most popular photo-sharing platforms, collected about 518 million of public photos in 2012.² Each photo comes with very useful information such as: tags, comments and likes from Flickr social network, number of views,

* Corresponding author.

E-mail addresses: igobrilhante@lia.ufc.br (I.R. Brilhante), jose.macedo@lia.ufc.br (J.A. Macedo), francomaria.nardini@isti.cnr.it (F.M. Nardini), raffaele.perego@isti.cnr.it (R. Perego), chiara.renso@isti.cnr.it (C. Renso).

¹ Part of this research has been conducted while visiting Federal University of Ceará, Fortaleza, Brazil.

² <http://www.flickr.com/photos/franckmichel/6855169886/>.

information about the user, timestamp, GPS coordinates of the place where the photo was taken. This allows us to roughly reconstruct the movements of users and their interests by analyzing the time-ordered sequence of their photos. However, the process of recognizing relevant Pols given such set of photos is not trivial due to the noise present in the data. User tags are in many cases missing, wrong, or irrelevant for our purposes (e.g., *me and Ann, travel to Europe, Easter 2012*). Moreover, information available may be sparse and characterized by a skewed distribution.

Fortunately, in Wikipedia,³ we can find that most entities of interest for tourism are described in a dedicated page from which we can extract: the (multilingual) name of the Pol, its precise geographic coordinates, the categories to which the Pol belongs according to a weak but robust ontology (i.e., the Pol is a church, a square, a museum, a historical building, a bridge, etc.). By spatially joining and re-conciliating tourists' photo albums and related information from Flickr with relevant Pols data extracted from Wikipedia pages, we can derive a knowledge base that represents the behavior of people visiting a given city.⁴ In this knowledge base the popularity of a Pol is estimated from the number of visitors that shot photos there, while from the timestamps of the first and last photos taken in a Pol we estimate the average time spent for the visit. Finally, we exploit the Wikipedia categories of the Pols visited by a given tourist to build her user profile. For example, when a user takes many pictures of churches and museums, we can infer a preference for cultural/historical attractions. Analogously, we can aggregate this information at the level of itinerary to build a profile for each frequent visiting pattern.

We address the problem of planning the visit to the city as a two-step process. First, given the profile of the user and the amount of time available for the visit, we formalize and address the TRIPCOVER problem: choosing the set of itineraries across the Pols that best fits user interest and respects the given time constraint. Then, the selected itineraries are joined in a sightseeing itinerary by means of a heuristic algorithm addressing the Trajectory Scheduling Problem (TRAJSP), a particular instance of Traveling Salesman Problem (TSP). The formalization of TRIPCOVER as an instance of the Generalized Maximum Coverage (GMC) problem can be found in Brilhante, Macedo, Nardini, Perego, and Renso (2013), while (Brilhante, Macedo, Nardini, Perego, & Renso, 2014) demos the capabilities of the TRIPBUILDER application.⁵ In this paper, we extend the previous research with the following relevant and original contributions:

- we detail our unsupervised method for mining common patterns of movements of tourists in a given geographic area. To this purpose we use: (i) Flickr, to gather public photos (and their metadata), (ii), Wikipedia to gather information regarding Pols in the given city, (iii) Google maps to estimate the time needed to move from a Pol to the next one in the sightseeing itinerary. The resulting knowledge base stores Pols, their popularity, the time needed in average to visit them, the categories for which each Pol is relevant, and the patterns of movement of tourists that visited them in the past. In order to assess our system, we report about the building of a knowledge base covering three Italian cities, which are important for tourism and guarantee variety and diversity in terms of size and the richness of public user-generated content available: Rome, Florence, and Pisa. The resulting knowledge base, available for download to favor the reproducibility of results, is analyzed and its characteristics are here discussed;
- we adopt the result discussed in Brilhante et al. (2013) where TRIPCOVER is defined and an approximate algorithm is proposed to compute the set of trajectories in the knowledge base that best fits user interests given a time budget. In this paper, we address the subsequent step that consists in processing this set of trajectories to build a schedule of the complete sightseeing itinerary over the tourist's agenda. To this end, we define TRAJSP as a variation of the TSP problem. It aims at finding the shortest path crossing all the trajectories in the TRIPCOVER solution. We investigate the TRAJSP problem and its peculiarities, which are exploited to devise an efficient and effective heuristic algorithm;
- finally, we report on several new experiments to evaluate effectiveness and efficiency of all the components of our system and show that our solution outperforms competitive baselines. In particular, we assess TRIPBUILDER performance in providing budgeted sightseeing itineraries made up of actual Pol patterns tailored to the specific preferences of the tourist.

The paper is structured as follows: Section 2 introduces the TRIPCOVER problem and the approximation algorithm used to solve it. Moreover, the TRAJSP problem is defined and addressed in Section 3. Section 4 details the unsupervised method that builds the knowledge base, while Section 5 presents the experiments we perform to assess the effectiveness and the efficiency of our solution. Finally, Section 6 discusses related work and Section 7 draws the conclusions of the work.

2. The TripCover problem

Let $\mathcal{P} = \{p_1, \dots, p_N\}$ be the set of Pols in our city. Each Pol p is univocally identified by its geographic coordinates, a name, a radius specifying its spatial extent, and a *relevance vector*, $\vec{v}_p \in [0, 1]^{|C|}$, measuring the normalized relevance of p w.r.t a set of categories C .

Symmetrically, let u be a user from the set $\mathcal{U}$, and $\vec{v}_u \in [0, 1]^{|C|}$ the *preference vector* stating the normalized interest of u for the categories in C . The preference vector can be explicitly given by the user, or implicitly learned. Without loss of generality,

³ <http://www.wikipedia.org>.

⁴ Thereinafter, we will consider cities as the destination targets of our users, although our technique is general and scale-independent.

⁵ <http://tripbuilder.isti.cnr.it/>.

in this paper we assume to know in advance the categories C , the relevance vectors $\vec{v}_p$, and the preference vectors $\vec{v}_u$ for all Pols and users.

Definition 1 (*User-Pol Interest*). Given a Pol p , its relevance vector $\vec{v}_p$, a user u , and the associated preference vector $\vec{v}_u$, we define the *User-Pol Interest* function as a the following function $\Gamma(p, u) : \mathcal{P} \times \mathcal{U} \rightarrow [0, 1]$:

$$\Gamma(p, u) = \alpha \cdot \text{sim}(\vec{v}_p, \vec{v}_u) + (1 - \alpha) \cdot \text{pop}(p)$$

where $\text{sim}(\vec{v}_p, \vec{v}_u) = \frac{\vec{v}_p \cdot \vec{v}_u}{\|\vec{v}_p\| \|\vec{v}_u\|}$ is the cosine similarity between the user preference and the Pol relevance vectors, $\text{pop}(p)$ is a function, ranging from 0 to 1, measuring the popularity of p , and $\alpha \in [0, 1]$ is a parameter controlling how much user preference and popularity of Pols have to be taken into account.

Definition 2 (*Pol History*). Given a user u and the Pols $\mathcal{P}$, the Pol history H_u of u is the temporally ordered sequence of m points of interest visited by u . Each Pol p of H_u is annotated with the two timestamps indicating the start time and the end time of the visit:

$$H_u = \langle (p_1, [t_{11}, t_{21}]), \dots, (p_m, [t_{1m}, t_{2m}]) \rangle$$

We can notice that having the start time and the end time we have an implicit representation of the time the user u has spent for her visit of p .

Definition 3 (*Trajectory*). Given a Pol History H_u and a time threshold δ , we define a trajectory T_u any subsequence of H_u

$$\langle (p_k, [t_{1k}, t_{2k}]), \dots, (p_{k+i}, [t_{1(k+i)}, t_{2(k+i)}]) \rangle$$

such that:

$$\begin{aligned} i &\geq 1 \\ t_{1k} - t_{2(k-1)} &> \delta, \quad \text{if } k > 1 \\ t_{1(k+i+1)} - t_{2(k+i)} &> \delta, \quad \text{if } (k+i) < m \\ t_{1(k+j)} - t_{2(k+j-1)} &\leq \delta, \quad \forall j \text{ s.t. } 1 \geq j \leq i. \end{aligned}$$

The intuition is that trajectories are sequences of Pols visited consecutively at the same “visit”. They are obtained by cutting the user Pol history where the time interval between the visit to two subsequent Pols is greater than a given threshold δ .

Example 1. Let John Smith be a tourist who visited the city of Rome for two-days. John Smith's Pol history consists in the temporally ordered sequence of Pols visited in the two days. As an example:

$$H_{\text{JohnSmith}} = \langle (\text{Colosseum, Tue}[09.00, 10.30]), (\text{Roman forum, Tue}[11.00, 12.00]), (\text{Spagna square, Tue}[14.30, 17.30]), (\text{St. Peter's Church, Wed}[10.00–11.00]), (\text{Vatican Museum, Wed}[11.10, 15.00]), (\text{Trevi Fountain, Wed}[16.30, 17.00]), (\text{Navona Square, Wed}[17.20, 18.00]), (\text{via Veneto, Wed}[18.35, 20.00]) \rangle.$$

By using a threshold of 5 h as trajectory splitting criterium, from $H_{\text{JohnSmith}}$ we obtain the following two trajectories:

$$\begin{aligned} T_{\text{JohnSmith}}^1 &= \langle (\text{Colosseum, Tue}[09.00, 10.30]), (\text{Roman forum, Tue}[11.00, 12.00]), (\text{Spagna square, Tue}[14.30, 17.30]) \rangle \\ T_{\text{JohnSmith}}^2 &= \langle (\text{St. Peter's Church, Wed}[10.00–11.00]), (\text{Vatican Museum, Wed}[11.10, 15.00]), (\text{Trevi Fountain, Wed}[16.30, 17.00]), (\text{Navona Square, Wed}[17.20, 18.00]), (\text{via Veneto, Wed}[18.35, 20.00]) \rangle \end{aligned}$$

The time interval between the visits to Spagna square and St. Peters Church is in fact the only interval larger than the given threshold.□

By applying the same temporal splitting criterium to all the Pol histories of users $\mathcal{U}$ we obtain the set $\mathcal{S} = \{S_1, \dots, S_M\}$ of relevant trajectories. Note that $\mathcal{S}$ results from a set-union operation disregarding timestamps. Finally, let $\rho(p) : \mathcal{P} \rightarrow \mathbb{R}$ be an estimate of the time needed to visit p , $\tau(p_i, p_j) : \mathcal{P} \times \mathcal{P} \rightarrow \mathbb{R}$ an estimate of the time a user needs to move from p_i to p_j , and $\vec{z} = (z_1, \dots, z_M)$ be the total traveling time associated with each of the M trajectories in $\mathcal{S}$, obtained by exploiting $\tau(\cdot, \cdot)$. We are now ready to formulate the TRIPCOVER problem, i.e., the problem of generating an optimal personalized itinerary given tourist's preferences and her budget in term of available time to spend in the city.

TripCover(B): Given a tourist u , a set of Pols $\mathcal{P}$, a time budget B , a set of trajectories $\mathcal{S}$, an User-Pol Interest function Γ , a cost function $\rho(p)$ and a vector $\vec{z}$. Find a subset of trajectories $\mathcal{S}^*$ of $\mathcal{S}$ that

$$\text{maximize } \sum_{i=1}^{|S|} \sum_{j=1}^{|P|} \Gamma(p_j, u) y_{ij} \quad (1)$$

$$\text{such that } \sum_{i=1}^{|S|} \sum_{j=1}^{|P|} \rho(p_j) y_{ij} + \sum_{i=1}^{|S|} z_i x_i \leq B \quad (2)$$

$$\sum_{i=1}^{|S|} y_{ij} \leq 1, \quad \forall j \in \{1, \dots, |P|\} \quad (3)$$

$$\sum_{i=1}^{|S|} x_i \geq \sum_{i=1}^{|S|} y_{ij}, \quad \forall j \in \{1, \dots, |P|\} \quad (4)$$

where

$$y_{ij} = \begin{cases} 1 & \text{if Pol } j \text{ in trajectory } i \text{ is selected;} \\ 0 & \text{otherwise.} \end{cases}$$

$$x_i = \begin{cases} 1 & \text{if trajectory } i \text{ is selected;} \\ 0 & \text{otherwise.} \end{cases}$$

Without loss of generality, we assume $\forall S' \in \mathcal{S}$, $\sum_{p \in S'} \Gamma(p, u) > 0$. In fact, if this would not hold for a given user u and some trajectories, these trajectories could be filtered out. The $\text{TRIPCOVER}(B)$ problem as formulated in (2) is an instance of the *Generalized Maximum Coverage* (GMC) problem that is proven to be NP-hard (Cohen & Katzir, 2008). The constraint (2) and (3) ensure the time budget is satisfied, and each selected Pol is associated with only one trajectory, respectively. Moreover, (4) guarantees a selected trajectory if a Pol is selected. In particular, given a tourist u , $\text{TRIPCOVER}(B)$ can be captured by the GMC formulation in the following way: (i) the bins in GMC represent the collection $\mathcal{S}$ of trajectories; (ii) the profit function $\Gamma(p, u)$ and the cost function $\rho(p)$ are bins-independent. They only depend on p and u . The $\text{TRIPCOVER}(B)$ problem is thus NP-hard. An efficient greedy approximation algorithm for solving the GMC problem that achieves an approximation ratio of $e/(e-1) + \epsilon$, $\forall \epsilon > 0$ is proposed in Cohen and Katzir (2008). We thus adapted this algorithm, whose source code has been kindly provided us by the authors, in order to take into account $\text{TRIPCOVER}(B)$ specific constraints.

3. The TRAJSP problem

Once the solution of a given TRIPCOVER instance is computed, the trajectories in S^* need to be scheduled on the user agenda. To this purpose, we model trajectory scheduling as a Traveling Salesman Problem (TSP) aimed at finding the shortest path crossing all these trajectories. In the classic TSP, the goal is to find the shortest path connecting a given set of geographical points. Here, the task is different as it is defined over a set S^* of disjoint trajectories, i.e., trajectories not sharing any Pol. We consider these trajectories as bi-directional paths representing tourists' behaviors that must be preserved in the final solution. Therefore we have to connect trajectories in a single sightseeing tour by only considering their terminal Pols (endPolS).

In the following, we formally define our trajectory scheduling problem (TRAJSP), we propose a local-search based algorithm to efficiently address it and, finally, we describe the simple approach used to schedule the final sightseeing tour on the user agenda.

Trajectory Scheduling Problem. Let $S^* \subseteq \mathcal{S}$ be a set of disjoint trajectories, $\mathcal{P}^*$ the set of endPolS, and $E = (e_{ij})$ the endPolS matrix where $e_{ij} = 1$ if i and j are endPolS of the same trajectory of S^* , 0 otherwise. Moreover, let C be the symmetric cost matrix where c_{ij} is the time needed to move from endPol i to endPol j . TRAJSP is defined as follows:

TRAJSP: Given the set of endPolS $\mathcal{P}^*$, the endPolS matrix E , and the cost matrix C , find the tour $\hat{\mathcal{P}}$ that:

$$\text{minimize } \sum_{i=1}^{|\mathcal{P}^*|} \sum_{j=i+1}^{|\mathcal{P}^*|} |c_{ij}| \eta_{ij} \quad (5)$$

$$\text{such that } \sum_{i=1}^{k-1} \eta_{ik} + \sum_{j=k+1}^{|\mathcal{P}^*|} \eta_{kj} = 1, \quad \forall k \in \{1, \dots, |\mathcal{P}^*|\} \quad (6)$$

$$\sum_{i,j \in S} \eta_{ij} + e_{ij} \leq |S| - 1, \quad (S \subset \mathcal{P}^*, |S| > 2), \quad i < j \quad (7)$$

$$\eta_{ij} \leq 1 - e_{ij}, \quad \forall i, j \ i \neq j \quad (8)$$

where

$$\eta_{ij} = \begin{cases} 1 & \text{if endPol } i \text{ is connected to endPol } j, \\ 0 & \text{otherwise.} \end{cases}$$

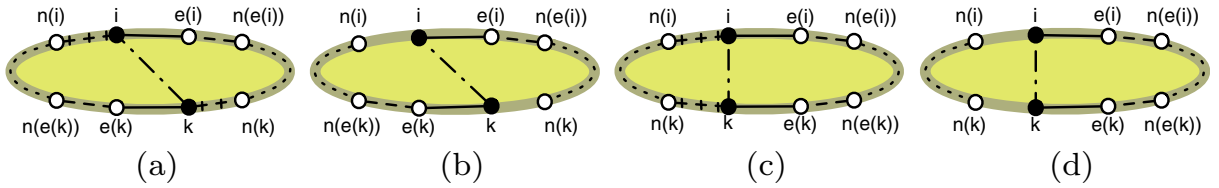


Fig. 1. Examples of how $op(i, k)$ modifies the tour according to $l(i, k)$, i.e., the minimum number of endPols connections between i and k .

In the above formulation, only the costs between different trajectories' endPols have to be considered for minimizing cost (5), while (6) and (7) impose constraints on the *degree* of each endPol and on *sub-tour elimination*, respectively (Rajesh Matai & Mittal, 2010). Note that the degree constraint in (6) is set to 1 since each endPol already has one fixed connection to the next Pol of the associated trajectory. The last constraint in (8) ensures that two endPols of the same trajectory are never connected together in the solution.

Although the number of possible solutions for $TRAJSP$ is lower than those of the corresponding TSP formulation,⁶ finding the exact solution to $TRAJSP$ is still infeasible even for instances involving a small number of trajectories (we have about 82 billions of possible solutions with only 12 trajectories). Hence, we address $TRAJSP$ by proposing a *Local Search* heuristics that starts from a (given or random) tour $\hat{P}$ connecting all trajectories in S^* , and then applying local changes to $\hat{P}$ by means of 2-OPT or 3-OPT strategies (Rajesh Matai & Mittal, 2010).

We now introduce an interesting property of the $TRAJSP$ problem that allows us to formalize how 2-OPT or 3-OPT strategies have to be applied. Given endPol i , let $e(i) = j$ be the other endPol of the same trajectory. Obviously $e(j) = i$ holds as well. Since we start from a tour connecting all the trajectories, each endPol i is connected to both $e(i)$ and to an endPol d of another trajectory in S^* . Let denote with $n(i)$ the connected endPol d . Note that only the link between endPols i and $n(i)$ can be modified, since the path between i and $e(i)$ is fixed by definition.

We now introduce a basic local-change operation $op(i, k)$ over two endPols i, k , such that $k \neq e(i)$. The local-change operation works by adding link (i, k) and removing links $(n(i), i)$ and $(n(k), k)$. It is clear that the application of $op(\cdot, \cdot)$ leads to a non-admissible solution for $TRAJSP$, since we remove two links of the tour while we add only one. We thus need to perform some additional changes to reconnect the two endPols that remain disconnected. The result below formalizes how such changes aimed at restoring the feasibility of the current solution have to be done. In particular, given two endPols i, k , it states that we can apply 2-OPT or 3-OPT strategies chosen according to the value of $l(i, k)$, a function returning the minimum number of endPols connections between i and k .

Lemma 1. Given a tour $\hat{P}$ and a local-change operation $op(i, k)$ performed over two endPols i, k such that $l(i, k) > 1$, if $l(i, k)$ is even, then a 2-OPT strategy needs to be applied to produce a feasible tour, a 3-OPT strategy otherwise.

Proof. Given two endPols i, k , $l(i, k)$ can be either even or odd. First, suppose $l(i, k)$ is even. It means that the minimum number of connections between the endPols i and k is even. As a consequence, starting from the link $(n(i), i)$ (crossed line), by walking anti-clockwise from i to k , we arrive at $(e(k), k)$ (solid line), or from $(i, e(i))$ (solid line) by walking clockwise, we arrive at $(k, n(k))$ (crossed line) as shown in Fig. 1(a).

Operation $op(i, k)$ removes $(n(i), i)$ and $(n(k), k)$. Moreover, it adds (i, k) (see Fig. 1(b)). Note that, at this point the tour is disconnected. To have a feasible tour, we need to add the new link $(n(i), n(k))$ that joins the two disconnected endPols. Because this process adds two links (i, k) , $(n(i), n(k))$ and removes two other links $(n(i), i)$, $(n(k), k)$, it corresponds to applying a 2-OPT strategy.

We now study the case $l(i, k)$ is odd. Starting from $(i, n(i))$, leads us to $(k, n(k))$ (crossed line). Moreover, from $(i, e(i))$ we arrive at $(k, e(k))$ (solid line). This configuration is shown in Fig. 1(c). Applying $op(i, k)$ to this particular case generates a different configuration (see Fig. 1(d)), i.e., one sub-tour (i.e., a closed path) from i to k , and one path from $n(i)$ and $n(k)$. To merge the sub-tour and the path, we need to remove one link from the sub-tour and connect it to the path, e.g., $n(i)$ and $n(k)$. So, we can remove $(e(i), n(e(i)))$. We then need to add $(e(i), n(i))$ and $(n(e(i)), n(k))$ to obtain a tour crossing all the endPols. Because we add three new links and remove three other ones, we applied a 3-OPT strategy. $\square$

Local search algorithm. From the above discussion, it is easy to devise a local search algorithm that iteratively optimizes a given tour by applying 2-OPT or 3-OPT strategies and stops after a fixed number of iterations or when it converges to a locally optimal solution.

Scheduling the tour on the user agenda. Given the sightseeing tour $\hat{P}$ computed for a given user and a time budget by our local search $TRAJSP$ algorithm, we schedule it on the user agenda by splitting the tour into the desired number m of slots (e.g., days). The solution involves identifying a starting endPol in the tour and assigning the successive Poles in $\hat{P}$ to the

⁶ We have exactly $(k-1)! \cdot 2^{(k-1)}$ solutions for k trajectories in the case of $TRAJSP$, and $(2k-1)!/2$ for the corresponding TSP formulation considering the two endPols of each trajectory.

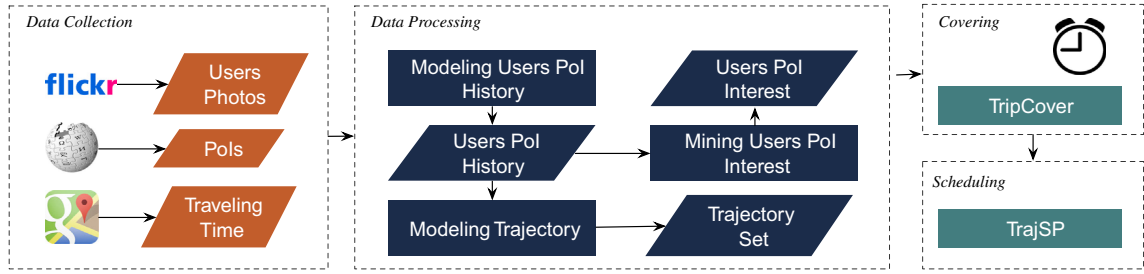


Fig. 2. Overview of the unsupervised process used to build the TripBuilder knowledge base.

current slot until the slot is filled and the next slot is considered. The choice of the starting endPol can be done in two different ways: either by removing the most “expensive” connection (e.g., in terms of traveling time) between two endPols of the tour or by taking into account the closest endPol to a given spatial position (e.g. user’s hotel).

4. Building the knowledge base

Fig. 2 depicts an overview of the *TRIPBUILDER* architecture. The component related to “Data Collection” retrieves relevant data from Flickr, Wikipedia, and Google Maps. The second component called “Data Processing” extracts the knowledge used to devise relevant Poles and model users’ visiting behaviors from data provided by the “Data Collection” component. Given a budget B , the third component “Covering” deals with the exploitation of the models and the knowledge base to compute the solution to the $\text{TRIPCOVER}(B)$ problem. The result is a set of trajectories in the chosen city on the basis of user interests and time budget that are finally scheduled on the user agenda by the fourth component “Scheduling”.

In order to assess *TRIPBUILDER* we generate – in a complete unsupervised process – a knowledge base covering three Italian cities which are important from a sightseeing point of view and guarantee variety and diversity in terms of size and richness of public user-generated content available for download: Rome, Florence, and Pisa. The generation of the knowledge base for each of the cities is a multi-steps process that we are going to detail in the following.

Poles. The first step is to identify the set of Poles in the target geographical region. Given the bounding box BB_{city} containing the city of interest, we download all the geo-referenced Wikipedia pages falling within this region. We assume each geo-referenced Wikipedia named entity, whose geographical coordinates falls into BB_{city} , to be a fine-grained Point of Interest. For each Pol, we retrieve its descriptive label, its geographic coordinates as reported in the Wikipedia page, and the set of categories the Pol belongs to. Categories are reported at the bottom of the Wikipedia page, and are used to link articles under a common topic. They form a hierarchy, although sub-categories may be a member of more than one category. By considering the set C of categories associated with all the Poles, we generate the normalized relevance vector of each Pol. We then perform a density-based clustering to group in a single Pol sightseeing entities which are very close one to each other.⁷ Clustering very close Poles is important since a tourist in a given place can enjoy all the attractions in the surroundings even if she do not take photos to all of them. Moreover, it aims at reducing the sparsity that might affect trajectory data. To cluster the Poles we use DBScan (Ester, Peter Kriegel, Sander, & Xu, 1996). To build our dataset, we set 1 as the minimum number of points and 200 m as ϵ . Finally, we obtain the relevance vector for the clustered Poles by considering the occurrences of each category in the members of the clusters and by normalizing the resulting vector. At the end of this first step we have the set $\mathcal{P} = \{p_1, \dots, p_N\}$ of Poles and the relevance vector $\vec{v}_p \in [0, 1]^{|C|}$ for each of these Poles in a fully automatic way by exploiting Wikipedia as an external source of knowledge.

Users and Pol histories. As second step we need a method for collecting users $\mathcal{U}$ and the long-term itineraries crossing the discovered Poles. We query Flickr to retrieve the metadata (user id, timestamp, tags, geographic coordinates, etc.) of the photos taken in the given area BB_{city} . The assumption we are making is that photo albums made by Flickr users implicitly represent sightseeing itineraries within the city. To strengthen the accuracy of our method, we retrieve only the photos having the highest geo-referenced accuracy given by Flickr.⁸ This process thus collects a large set of geo-tagged photo albums taken by different users within BB_{city} . We preliminarily discard photo albums containing only one photo. Then, we spatially match the remaining photos against the set of Poles previously collected. We associate a photo to a Pol when it has been taken within a circular buffer of a given radius having the Pol as its center. To build our dataset, we empirically set it to 100 m this radius. Note that in order to deal with clustered Poles, we consider the distance of the photo from all constituent members: in the case the photo falls within the circular region of at least one of the members, it is assigned to the clustered Pol. Moreover, since several photos by the same user are usually taken close to the same Pol, we consider the timestamps associated with the first and last of these photos as the starting and ending time of the user visit to the Pol. The Pol visiting time $\rho(p)$ is then estimated by

⁷ Consider for example the beautiful marble statues in the *Loggia dei Lanzi* in Florence which are only a few meters far one from each other but have a distinct dedicated page in Wikipedia.

⁸ <http://www.flickr.com/services/api/flickr.photos.search.html>.

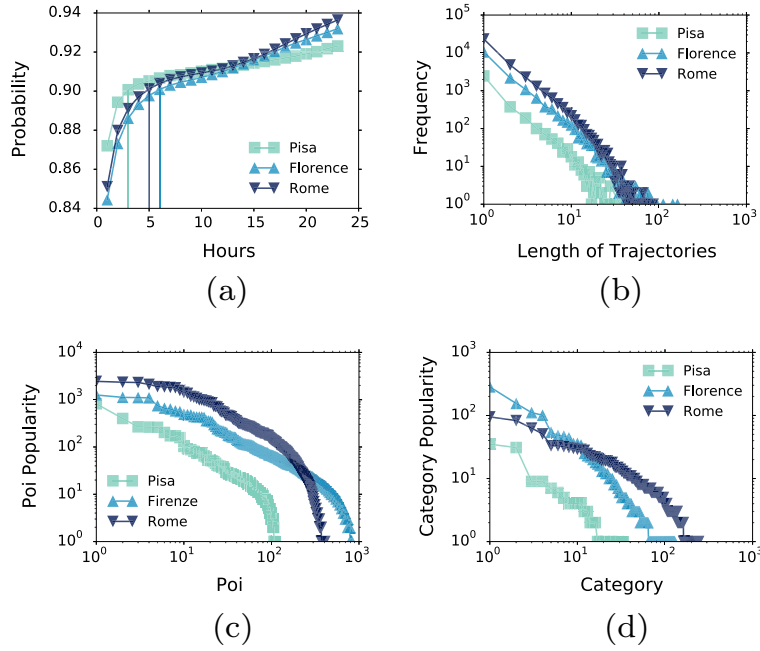


Fig. 3. Plot (a) shows the probability distribution of the inter-arrival time for pairs of consecutive photos taken in Rome, Florence, and Pisa. The vertical lines highlight the time thresholds corresponding to $P(x \leq \delta) = 0.9$. Plots (b)–(d) show instead the distribution of trajectories length (as number of POIs crossed), the popularity of POIs and the popularity of categories in the three datasets.

Table 1

Statistics regarding the three cities in our dataset.

City	Pols	Users	Photos	Trajectories	Traj. per Poi (avg.)
Pisa	112	1825	18,170	3430	7.20
Florence	891	7049	102,888	16,522	5.39
Rome	490	13,772	234,616	35,522	20.51

computing for each Poi the average of these times. Moreover, the popularity of each Poi is computed as the number of distinct users that take at least one photo in its circular region. The above process allows us to generate the set of users $\mathcal{U}$, their Poi history, and estimate for the popularity and visiting time of each Poi. Finally, the preference vector for each user is built by summing up and normalizing the relevance vectors of all the POIs occurring in her Poi history.

Trajectories. In order to build the set $\mathcal{S}$ of trajectories we split users' Poi histories as detailed in Definition 2. To choose the splitting threshold δ , we derive the users' wisdom-of-crowds behavior by analyzing the inter-arrival time of each pair of consecutive photos taken in different POIs. Therefore, for each city we compute the distribution of probability of the inter-arrival time $P(x \leq \delta)$ of pairs of consecutive photos. Then, we devise the time threshold δ such that $P(x \leq \delta) = 0.9$. Fig. 3(a) shows the distributions of probability of inter-arrival times, i.e., $P(x \leq \delta)$, for all the pairs of consecutive photos in each dataset. Results show that while for Rome and Florence the resulting δ is about 5 and 6 h respectively, for the smallest city of Pisa it decreases to about 3 h.

Traveling time estimation. An important aspect of TRIPBUILDER is that we recommend sightseeing tours fitting the available time budget and not just the set of POIs to be visited. The sightseeing tour building step should therefore consider not only the Poi visiting time $\rho(p)$ but also the time $\tau(\cdot, \cdot)$ needed to move between consecutive POIs in the itinerary. Since measuring intra-Poi moving time from the photo albums resulted to be inaccurate for not popular POIs, we resort to an external service. Given a pair (p_i, p_j) of POIs in a trajectory, we estimate $\tau(p_i, p_j)$ by querying Google Maps for the walking time between the POIs. Naturally, this is an approximation since several variations may happen: the user having a car, using public transportation, taking a taxi. However, our method is parametric to these aspects, and the system can be easily adapted to consider the different choices. Moreover, most POIs in our sightseeing cities are actually at walking distances.

Datasets statistics. Table 1 shows the main characteristics of the three datasets. The second column reports the number of POIs for each of the three cities. Note that these numbers refer to the result of the clustering phase, while the number of entities extracted from Wikipedia is 124, 1022, and 671 for Pisa, Florence and Rome, respectively. Furthermore, columns "Users" and "Photos" report the number of distinct users and photos retrieved from Flickr. Finally, column "Trajectories"

Table 2

Top-3 most popular Pols and Categories in Pisa, Florence, and Rome. We also report three examples of trajectories per city extracted from the dataset.

	Pisa	Florence	Rome
Pols	Leaning Tower Miracoli Square Battistero di San Giovanni	Loggia dei Lanzi Ponte vecchio Battistero di San Giovanni	Colosseum Fontana di Trevi Pantheon
Categories	Churches Palaces Museums	Palaces Churches Architecture	Churches Campitelli Quarter Titular Churches
Trajectories	Battistero di San Giovanni, Miracoli Square, Leaning Tower Miracoli Square, Museo dell'Opera del Duomo, Leaning Tower Ebraic Cemetery, Battistero di San Giovanni, Leaning Tower	Battistero di San Giovanni, Porta della Mandorla, Campanile di Giotto Palazzo Vecchio, Galleria degli Uffizi, Piazza della Signoria Ponte Vecchio, Forte Belvedere, Piazza della Signoria	Cappella Sistina, Basilica di San Pietro in Vaticano, Obelisco Vaticano Colosseum, Arco di Costantino, Foro Romano Colosseum, Foro Romano, Pantheon

reports the number of trajectories crossing at least two Pols, while column “Traj. per Pol (avg.)” reports the average number of trajectories crossing each Pol.

Fig. 3(b)–(d) show three plots regarding the characteristics of the three datasets that have been made available for download to favor the reproducibility of experiments.⁹ A general consideration common to all the three figures regards the skewness of the distributions that are plotted in log–log scale. In particular, Fig. 3(b) reports the distribution of trajectory length expressed in term of the number of crossed Pols. We can see that the slope of the three curves is very similar, only absolute values vary as expected. Note that the most frequent trajectory length in the plot is just 1. These are obviously noisy trajectories, corresponding to the cases in which we can match only one single Pol to a photo album. These spurious trajectories are maintained in the datasets only for their contribution to the measure of Pol popularity, but do not belong to the set S of trajectories used by TRIPBUILDER.

Fig. 3(c) shows the distribution of Pol popularity in the three cities involved. Even in this case we can note strong similarities in the distributions, although the curves of Rome and Florence cross in the tail. This happens for a peculiarity in these two datasets: while we have more users and photos for Rome than for Florence, the opposite holds for the number of Pols reported in the X axis.

The last plot reported in Fig. 3(d) shows the popularity of categories associated with the Pols. Even in this case we have a power-law distribution with a few very popular categories and most categories associated with relatively few Pols. Wikipedia categories form in fact a (weak) hierarchy, and the most general topics are associated with many pages, while sub-categories are relevant only for precisely identified subsets of homogeneous Pols.

Finally, Table 2 report the top-3 most popular Pols and categories in Pisa, Florence and Rome. We also report three examples per city of trajectories extracted from the dataset. Note that a popular Pol in Florence and Pisa is the baptistery which is entitled, in both cities, to Saint John the Baptist.

5. Experiments

We now assess the effectiveness of TRIPBUILDER in: (i) selecting a set of trajectories of interest for a given user (TRIPCOVER), and (ii) scheduling that set on the user agenda (TRAJSP). This is done by comparing its performance with those obtained by competitive baseline by means of evaluation metrics that consider the actual behavior of test users as mined from Flickr. Moreover, we present an evaluation of the efficiency of the TRIPBUILDER framework together with a detailed evaluation of both TRIPCOVER and TRAJSP solutions.¹⁰

We conduct our experiments on the three datasets of Pisa, Florence, and Rome by varying the time budget and the parameter α affecting the contribution of Pols/user-similarity and Pol-popularity to user profit. Moreover, we perform two different set of experiments that differ for the methodology used to choose the test users:

- *Random selection.* Here the set of users used to assess TRIPBUILDER performance is randomly chosen. In particular, we consider for all the three cases a set of 100 test users randomly selected among the visitors having a Poi history longer than 10, 15, and 20 Pols for Pisa, Florence and Rome, respectively. The threshold on the length of the Poi history is set in order to be able to vary in a significant range the time budgets. This is because it is not feasible to evaluate a personalized 4-days itinerary in Rome with test users that actually visited only a few popular Pols during a single day of visit. By using the above cutoff values, the users among which the 100 test users were chosen are 153, 679, and 930 in Pisa, Florence, and Rome, respectively.

⁹ Interested readers can download the datasets from the URL: <https://github.com/igobrilhante/TripBuilder>.

¹⁰ All the experiments have been conducted on a Ubuntu Linux box with two Intel®Xeon®E5520 CPUs and 32 GB of RAM.

- **Profile-based selection.** Here we select the test users among users who actually visited at least two of the three cities. In particular, given a user visiting two cities A and B , we used the preference vector obtained from the Pols visited in city A to generate the personalized sightseeing tour in city B and vice versa. In this way we avoid any possible bias to the specific categories used in the Wikipedia pages of a given city.

Effectiveness – TripCover. We compare the effectiveness of TRIPBUILDER in selecting a budgeted set of trajectories of interest for a given user against the following baselines:

- **Trajectory Popularity (Tpop).** This baseline builds the tour by taking into account the normalized popularity of the trajectories in S computed as the sum of the popularity of the constituent Pols divided by the length of the trajectory. It greedily adds to the visiting plan the most popular trajectories until the time budget is reached.
- **Trajectory Personalized Profit (Tppro).** Given the preference vector of a tourist, this baseline sorts the trajectories in S by decreasing normalized user/Pol similarity. Such trajectory score is computed as the sum of user/Pol similarities of all the Pols in the trajectory divided by the trajectory length. The baseline builds the personalized itinerary by adding once at a time the trajectories having the highest profit for the specific tourist until the total time budget is reached.

Experiments are conducted by providing to TRIPBUILDER and the baseline algorithms the preference vector of each one of the test users in each city, along with a time budget varying in the range 1, 2, and 4 days (1/2, 1 day in the case of the small city of Pisa).¹¹ We evaluate the performance of the three methods by means of the metrics defined in Table 3. Moreover, we also employ *recall* (Baeza-Yates & Ribeiro-Neto, 1999), a well-known IR metrics that in our settings measure the ability of the methods to predict Pols and categories that match actual Pol histories of the users in the test set.

Random Selection. Tables 4–6 report the effectiveness measured for the recommended sightseeing tours on the basis of the metrics defined in Table 3 in Pisa, Florence, and Rome, respectively. In particular, each table details the average per-user performance and its standard deviation for each previously-defined metrics, and highlights in bold fonts the best per-budget figures. The first observation from the results is that, despite of their simplicity, both popularity (Tpop) and profit-based (Tppro) greedy strategies perform well, thus forming competitive baselines. In terms of Personal Profit Score (S_u^{pro}), our solution improves the baselines up to 86% in Pisa with an absolute improvement ΔS_u^{pro} of 0.30 (Tpop vs. TRIPBUILDER, $\alpha = 0.75$), 178% in Florence with an improvement ΔS_u^{pro} of 0.203 (Tpop vs. TRIPBUILDER, $\alpha = 1$), and 213% in Rome with an improvement ΔS_u^{pro} of 0.382 (Tpop vs. TRIPBUILDER, $\alpha = 0.75$). In terms of S_u^{pro} , TRIPBUILDER outperforms the baselines showing a behavior which is sensitive to the parameter α .

Results in Pisa (see Table 4) compared to Tppro demonstrate that our approach is better for 1/2-day budget with 6% of improvement, while Tpop is better for 1-day budget, but it improves only 1%. It is worth highlighting two situations: (1) when $\alpha = 0$, TRIPBUILDER works by considering only the popularity (as Tpop) showing a performance in terms of S_u^{pro} that is similar to the Tpop baseline; (2) when $\alpha = 1$, TRIPBUILDER considers only users' interest, and its performance becomes comparable with Tppro. We conclude that α plays an important role in TRIPBUILDER to balance the contribution of users' profit/interest and Pol's popularity. For instance, the highest S_u^{pro} score in Pisa for the time budget of 1 day can be found when $\alpha = 0.75$ (i.e., 0.79). Moreover, TRIPBUILDER builds tours that increase the Visiting Time Score (S^{vt}) up to 36 min in Pisa ($\Delta S^{vt} = 0.05$), about 279 min in Florence, and approximately 9.2 h in Rome. Therefore, our algorithm is able to suggest itineraries that better match users' preferences w.r.t the baselines. In addition, TRIPBUILDER works by favoring higher visiting time while contributing to lower intra-Pol movement time in its solutions.

In terms of Visiting Time Score (S^{vt}), the higher it is, the more relevant the itinerary can be considered for the user, since it is likely that she prefer to spend time visiting the Pols than moving between them. As TRIPCOVER takes this factor into account (as a cost), it tends to exploit trajectories containing Pols closer to each other to maximize the user profit. Consequently, we claim that TRIPBUILDER is able to build tours that globally maximize S^{vt} . We can see from the results in the tables that TRIPBUILDER uses more appropriately the time budget. The difference in terms of S^{vt} increases when larger budgets are considered. Moreover, this phenomenon is even more highlighted when dealing with larger cities. Indeed, in the case of Pisa, the three algorithms (i.e., Tpop, Tppro, TRIPBUILDER) have quite similar S^{vt} , with slight gains for TRIPBUILDER (from 30 to 34 min, with $\alpha = 0.75$). In the case of larger cities, i.e., Florence and Rome, TRIPBUILDER remarkably outperforms the baselines. The rationale behind this result can be that in larger cities the intra-Pol traveling time tends to impact more the S^{vt} metrics.

In terms of Popularity Score (S^{pop}), results achieved with $\alpha = 0$ confirm that TRIPBUILDER always outperforms Tpop. Moreover, figures for TRIPBUILDER decreases for increasing values of α as expected.

In terms of Pols and categories recall, all algorithms get at least 69% of the relevant Pols and 81% of the categories for Pisa. Regarding categories recall in Pisa (see Table 4), TRIPBUILDER and Tpop present similar results, while both of them outperform Tppro. Moreover, looking at Pols recall, TRIPBUILDER gets better results than the baselines (with $\alpha = 0.5$): 92.7% compared with 89.4% of Tpop and 90.9% of Tppro for the 1-day time budget. Comparing the results for Florence and Rome, we observe that TRIPBUILDER outperforms in both recall figures the baselines depending on α setting. Moreover, when larger budgets are employed (2 and 4-days budget), it always outperforms the baselines independently from the value of α (see

¹¹ We assume the normal daily activity of a tourist in a city to be of twelve hours. Our solution is, however, completely agnostic w.r.t. the daily agenda and works with tourist-provided agenda defining different time slots as well.

Table 3

Evaluation metrics used to assess the effectiveness of TRIPBUILDER in selecting a set of trajectories of interest for a given user.

Personal Profit Score	$S_u^{pro}(S^*) = \frac{\sum_{p \in S^*} \text{sim}(\vec{v}_p, \vec{v}_u)}{\sum_{p \in P} \text{sim}(\vec{v}_p, \vec{v}_u)}$	Given a user u and a set of trajectories S^* , S_u^{pro} is computed as the sum of the profits of the Pols in S^* divided by the sum of the profits of all the Pols. The user profit for a Pol (i.e., $\text{sim}(\vec{v}_p, \vec{v}_u)$) is the cosine similarity between user preferences and Pol relevance vectors (see Definition 1)
Visiting Time Score	$S^{vt}(S^*) = \sum_{p \in S^*} \rho(p)/B$	Given a set of trajectories S^* , this score is computed as the sum of the visiting times for the Pols in S^* normalized by the time budget B . Given a time budget, it assumes that high scored tours result to be interesting since they favor the time to enjoy attractions with respect to the intra-Pols moving time
Popularity Score	$S^{pop}(S^*) = \sum_{p \in S^*} \text{pop}(p)$	Given a set of trajectories S^* , this score is computed by summing the popularity of the Pols in S^* . Note that the popularity $\text{pop}(p)$ of a given Pol p is normalized over the sum of the popularity of all the Pols. As a consequence, $\sum_{p \in P} \text{pop}(p) = 1$

Tables 5 and 6). This behavior is due to the capability of TRIPBUILDER of building tours with a higher Visiting Time Score within the time budgets. Consequently, more Pols relevant for the specific user are likely to be visited. Moreover, we can see that the α parameter allows to fit the expectations of the user. For small values of α , results report higher recall values because trajectories crossing popular Pols are preferred. When α increases, recall decreases because unexpected Pols in the selected trajectories fitting the user interests are suggested. We believe these trajectories may constitute a source of serendipitous recommendations. Finally, the low standard deviations associated with the results in Tables 4–6 prove that the performance of the three techniques are stable w.r.t. different users' profiles.

Profile-based Selection. In these experiments we select test users who visited two different cities A and B . We employ their preferences in city A to build and evaluate tours in city B and vice versa. Due to a lack of space, we report only the results obtained by considering Rome as target city B and Florence as city A . However, the results obtained considering different pairs of cities are similar thus confirming the behavior identified. Among the 2224 users visiting both Florence and Rome, we randomly chose 100 users having Pol histories longer than the thresholds discussed above. It is worth noting that for these experiments we need to “uniform” the categories of the Pols in the three cities, since Wikipedia provides different categories for each city. Therefore, we exploit the general categorization used within the TRIPBUILDER Web application available online¹²: *Architecture, Arts, Churches, Entertainment, Monuments, Museums, Nature & Landmarks*.

The results of the experiments are reported in Table 7. As we can see, the results confirm the trends reported for the previous experiments showing that TRIPBUILDER remarkably outperforms the baselines. Moreover, the similarity of these results with those obtained in the tests using random selection confirm that the performance of TRIPBUILDER is not affected by the “over-specific” categorization used in Wikipedia pages. Finally, it is worth noting that the recall figures computed at the category level are remarkably higher than in the previous experiments. This is motivated by the lower number of categories used to conduct the experiments: covering the whole set of categories with a sightseeing itinerary is in this case much simpler. It is also worth noting that we do not report results of TRIPBUILDER with $\alpha = 0$ here because we are conducting a profile-based experiment. TRIPBUILDER with $\alpha = 0$ does not exploit personalization. For this reason, we do not consider it in this analysis.

Effectiveness – TRAJSP. We now evaluate the effectiveness of TRIPBUILDER in scheduling the candidate set of trajectories on the user agenda. We evaluate effectiveness by considering the *average path cost*, the ratio between the cost of the TRAJSP solution – i.e., the length of the connections between trajectories endPols – over the total budget available (see Fig. 4). The lower the ratio is, the better TRIPBUILDER employs the time by minimizing the connections when solving the TRAJSP instance. Our *Local Search* algorithm is compared with two baselines: (i) *Random*, which builds the sightseeing tours by randomly connecting the endPols of the trajectories; and (ii) *Nearest Trajectory*, which is an adaptation of the well-known “nearest neighbor” TSP greedy heuristic (Rajesh Matai & Mittal, 2010) that greedily constructs the solution by always selecting the trajectory with the closest endPol. The results reported are referred to the whole set of 100 users in the test set. However, since we are evaluating heuristic approaches that might be very sensitive to the starting conditions, we ran each experiment 5 times and averaged the results achieved. The stop condition for our TRAJSP local search algorithm used in all tests is reaching 1000 iterations or 100 iterations without improvement in the solution cost.

Results show that the Random baseline immediately diverges from the other competitors getting the worst paths. Nearest Trajectory performs better than Random and it is always able to provide an average connection cost lower than 0.3. On the other hand, our Local Search algorithm always outperforms both Random and Nearest Trajectories, with a notable improvement in all the three cities considered. In particular, it provides solutions having an average connection cost lower than 0.2. In other words, TRIPBUILDER is able to address TRAJSP even in big cities like Rome by adding at most 20% of the total time budget for inter-trajectory connections.

¹² <http://tripbuilder.isti.cnr.it/>.

Table 4

Random Selection: average effectiveness of TRIPBUILDER and the baselines in Pisa.

	Days	Recall (Pols)	Recall (Cat.)	S_u^{pro}	S^{rt}	S^{pop}
Tpop	1/2	0.792 (± 0.17)	0.951 (± 0.08)	0.348 (± 0.05)	0.639 (–)	0.832 (–)
	1	0.894 (± 0.10)	0.989 (± 0.04)	0.655 (± 0.06)	0.643 (–)	0.914 (–)
Tppro	1/2	0.697 (± 0.19)	0.819 (± 0.18)	0.609 (± 0.11)	0.648 (± 0.04)	0.468 (± 0.14)
	1	0.909 (± 0.10)	0.966 (± 0.07)	0.804 (± 0.06)	0.643 (± 0.02)	0.780 (± 0.10)
TRIPBUILDER ($\alpha = 0.0$)	1/2	0.796 (± 0.16)	0.983 (± 0.04)	0.400 (± 0.05)	0.731 (–)	0.836 (–)
	1	0.861 (± 0.11)	0.989 (± 0.04)	0.584 (± 0.05)	0.640 (–)	0.887 (–)
TRIPBUILDER ($\alpha = 0.25$)	1/2	0.824 (± 0.14)	0.946 (± 0.09)	0.619 (± 0.08)	0.744 (± 0.03)	0.717 (± 0.09)
	1	0.914 (± 0.09)	0.990 (± 0.03)	0.751 (± 0.05)	0.692 (± 0.02)	0.899 (± 0.02)
TRIPBUILDER ($\alpha = 0.5$)	1/2	0.795 (± 0.16)	0.917 (± 0.12)	0.641 (± 0.09)	0.736 (± 0.03)	0.636 (± 0.12)
	1	0.927 (± 0.08)	0.993 (± 0.02)	0.779 (± 0.06)	0.692 (± 0.02)	0.888 (± 0.04)
TRIPBUILDER ($\alpha = 0.75$)	1/2	0.772 (± 0.15)	0.903 (± 0.12)	0.646 (± 0.09)	0.731 (± 0.03)	0.598 (± 0.14)
	1	0.926 (± 0.08)	0.991 (± 0.03)	0.790 (± 0.06)	0.689 (± 0.02)	0.879 (± 0.05)
TRIPBUILDER ($\alpha = 1.0$)	1/2	0.769 (± 0.16)	0.895 (± 0.13)	0.644 (± 0.09)	0.731 (± 0.03)	0.579 (± 0.14)
	1	0.918 (± 0.09)	0.991 (± 0.03)	0.788 (± 0.06)	0.693 (± 0.02)	0.861 (± 0.05)

Table 5

Random Selection: average effectiveness of TRIPBUILDER and the baselines in Florence.

	Days	Recall (Pols)	Recall (Cat.)	S_u^{pro}	S^{rt}	S^{pop}
Tpop	1	0.565 (± 0.17)	0.905 (± 0.08)	0.114 (± 0.04)	0.601 (–)	0.591 (–)
	2	0.708 (± 0.17)	0.952 (± 0.06)	0.214 (± 0.04)	0.607 (–)	0.732 (–)
	4	0.822 (± 0.13)	0.982 (± 0.04)	0.359 (± 0.04)	0.609 (–)	0.836 (–)
Tppro	1	0.359 (± 0.15)	0.720 (± 0.17)	0.203 (± 0.06)	0.648 (± 0.05)	0.249 (± 0.07)
	2	0.515 (± 0.15)	0.864 (± 0.10)	0.338 (± 0.07)	0.637 (± 0.04)	0.382 (± 0.07)
	4	0.719 (± 0.13)	0.961 (± 0.05)	0.529 (± 0.06)	0.631 (± 0.03)	0.602 (± 0.05)
TRIPBUILDER ($\alpha = 0.0$)	1	0.672 (± 0.17)	0.957 (± 0.06)	0.196 (± 0.04)	0.792 (–)	0.689 (–)
	2	0.785 (± 0.16)	0.972 (± 0.05)	0.325 (± 0.04)	0.722 (–)	0.803 (–)
	4	0.860 (± 0.10)	0.986 (± 0.03)	0.453 (± 0.03)	0.706 (–)	0.869 (–)
TRIPBUILDER ($\alpha = 0.25$)	1	0.572 (± 0.15)	0.912 (± 0.08)	0.316 (± 0.05)	0.723 (± 0.03)	0.457 (± 0.06)
	2	0.749 (± 0.13)	0.964 (± 0.05)	0.457 (± 0.05)	0.691 (± 0.02)	0.663 (± 0.05)
	4	0.874 (± 0.09)	0.986 (± 0.03)	0.594 (± 0.04)	0.694 (± 0.02)	0.826 (± 0.02)
TRIPBUILDER ($\alpha = 0.5$)	1	0.554 (± 0.15)	0.903 (± 0.09)	0.317 (± 0.05)	0.719 (± 0.03)	0.435 (± 0.06)
	2	0.742 (± 0.13)	0.962 (± 0.05)	0.458 (± 0.05)	0.689 (± 0.02)	0.646 (± 0.05)
	4	0.869 (± 0.10)	0.986 (± 0.03)	0.595 (± 0.04)	0.692 (± 0.02)	0.822 (± 0.03)
TRIPBUILDER ($\alpha = 0.75$)	1	0.548 (± 0.15)	0.897 (± 0.09)	0.317 (± 0.05)	0.717 (± 0.03)	0.427 (± 0.06)
	2	0.741 (± 0.13)	0.961 (± 0.05)	0.458 (± 0.05)	0.688 (± 0.02)	0.641 (± 0.05)
	4	0.868 (± 0.10)	0.986 (± 0.03)	0.596 (± 0.04)	0.692 (± 0.02)	0.820 (± 0.03)
TRIPBUILDER ($\alpha = 1.0$)	1	0.546 (± 0.15)	0.897 (± 0.09)	0.317 (± 0.05)	0.716 (± 0.03)	0.424 (± 0.06)
	2	0.736 (± 0.13)	0.962 (± 0.05)	0.458 (± 0.05)	0.686 (± 0.02)	0.638 (± 0.05)
	4	0.866 (± 0.10)	0.986 (± 0.03)	0.596 (± 0.04)	0.689 (± 0.02)	0.817 (± 0.03)

Efficiency. We now report results of experiments conducted to evaluate the efficiency of TRIPBUILDER as a function of the time budget available for the three cities. In particular, we consider time budgets of 1, 2, and 4 days (1/2, 1 days in the case of the small city of Pisa), and, for each instance of the experiment (time budget, city), we run the algorithm for all the 100 distinct users in the test set and all the values of α used in the experiments reported in Tables 4–6. Fig. 5 shows the average runtime of TRIPBUILDER along with standard deviation. Moreover, the plot highlights the contributions of TRIPCOVER (bottom bars) and of our TRAJSP local search algorithm (upper bars) to the overall runtime. It is worth noting that TRIPBUILDER always completes the tour building process in a few seconds. Moreover, the lower the number of trajectories in the dataset, the more similar the runtime is for all budgets. On the other hand, long time budgets (e.g. 4 days) and large datasets impact the total runtime, as expected. Results also reveals low standard deviation of the average runtime thus confirming that the whole process is feasible for online applications.

Fig. 6 details the average runtime of the techniques employed to solve TRAJSP. As before, we compute the efficiency by averaging the runtime obtained in 5 runs for each one of the 100 users of the test set. The Random technique obviously has the best runtime performance even if it is not effective in providing solutions as good as the other approaches do. For

Table 6

Random Selection: average effectiveness of TRIPBUILDER and the baselines in Rome.

	Days	Recall (Pols)	Recall (Cat.)	S_u^{pro}	S^{rt}	S^{pop}
Tpop	1	0.578 (± 0.15)	0.881 (± 0.07)	0.179 (± 0.03)	0.471 (–)	0.642 (–)
	2	0.795 (± 0.13)	0.949 (± 0.05)	0.403 (± 0.03)	0.463 (–)	0.836 (–)
	4	0.934 (± 0.07)	0.991 (± 0.02)	0.675 (± 0.03)	0.485 (–)	0.953 (–)
Tppro	1	0.428 (± 0.12)	0.708 (± 0.14)	0.381 (± 0.06)	0.456 (± 0.04)	0.281 (± 0.07)
	2	0.642 (± 0.13)	0.872 (± 0.09)	0.596 (± 0.06)	0.443 (± 0.03)	0.505 (± 0.08)
	4	0.894 (± 0.09)	0.979 (± 0.03)	0.824 (± 0.04)	0.447 (± 0.02)	0.813 (± 0.04)
TRIPBUILDER ($\alpha = 0.0$)	1	0.749 (± 0.13)	0.937 (± 0.05)	0.369 (± 0.03)	0.786 (–)	0.800 (–)
	2	0.906 (± 0.09)	0.980 (± 0.03)	0.603 (± 0.04)	0.754 (–)	0.927 (–)
	4	0.950 (± 0.05)	0.991 (± 0.01)	0.742 (± 0.03)	0.635 (–)	0.961 (–)
TRIPBUILDER ($\alpha = 0.25$)	1	0.680 (± 0.12)	0.878 (± 0.07)	0.558 (± 0.04)	0.735 (± 0.02)	0.563 (± 0.05)
	2	0.889 (± 0.08)	0.978 (± 0.02)	0.757 (± 0.04)	0.723 (± 0.01)	0.837 (± 0.03)
	4	0.966 (± 0.04)	0.995 (± 0.01)	0.860 (± 0.03)	0.640 (± 0.02)	0.955 (± 0.01)
TRIPBUILDER ($\alpha = 0.5$)	1	0.663 (± 0.13)	0.854 (± 0.09)	0.561 (± 0.04)	0.732 (± 0.02)	0.528 (± 0.06)
	2	0.885 (± 0.08)	0.974 (± 0.03)	0.760 (± 0.04)	0.720 (± 0.01)	0.809 (± 0.04)
	4	0.968 (± 0.04)	0.995 (± 0.01)	0.865 (± 0.03)	0.640 (± 0.02)	0.954 (± 0.01)
TRIPBUILDER ($\alpha = 0.75$)	1	0.654 (± 0.13)	0.841 (± 0.09)	0.561 (± 0.04)	0.731 (± 0.02)	0.513 (± 0.06)
	2	0.880 (± 0.08)	0.973 (± 0.03)	0.761 (± 0.04)	0.718 (± 0.02)	0.798 (± 0.05)
	4	0.967 (± 0.04)	0.995 (± 0.01)	0.867 (± 0.03)	0.639 (± 0.02)	0.954 (± 0.01)
TRIPBUILDER ($\alpha = 1.0$)	1	0.651 (± 0.13)	0.839 (± 0.09)	0.561 (± 0.04)	0.731 (± 0.02)	0.505 (± 0.06)
	2	0.880 (± 0.08)	0.970 (± 0.03)	0.762 (± 0.04)	0.718 (± 0.02)	0.792 (± 0.05)
	4	0.967 (± 0.04)	0.995 (± 0.01)	0.867 (± 0.03)	0.638 (± 0.02)	0.953 (± 0.01)

Table 7

Profile-based selection: average effectiveness of TRIPBUILDER and the baselines in Rome obtained by exploiting the profiles from Florence visits.

	Days	Recall (Pols)	Recall (Cat.)	S_u^{pro}	S^{rt}	S^{pop}
Tpop	1	0.650 (± 0.14)	0.838 (± 0.08)	0.187 (± 0.02)	0.471 (–)	0.642 (–)
	2	0.859 (± 0.10)	0.975 (± 0.06)	0.407 (± 0.02)	0.463 (–)	0.836 (–)
	4	0.955 (± 0.06)	1.000 (–)	0.676 (± 0.02)	0.485 (–)	0.953 (–)
Tppro	1	0.349 (± 0.09)	0.767 (± 0.12)	0.160 (± 0.05)	0.416 (± 0.03)	0.356 (± 0.05)
	2	0.526 (± 0.11)	0.791 (± 0.11)	0.286 (± 0.08)	0.427 (± 0.02)	0.531 (± 0.06)
	4	0.768 (± 0.10)	0.853 (± 0.09)	0.511 (± 0.10)	0.414 (± 0.02)	0.772 (± 0.05)
TRIPBUILDER ($\alpha = 0.25$)	1	0.627 (± 0.09)	0.883 (± 0.10)	0.389 (± 0.05)	0.787 (± 0.01)	0.624 (± 0.03)
	2	0.824 (± 0.08)	1.000 (–)	0.630 (± 0.04)	0.739 (± 0.01)	0.842 (± 0.02)
	4	0.921 (± 0.05)	1.000 (–)	0.790 (± 0.03)	0.642 (± 0.02)	0.937 (± 0.01)
TRIPBUILDER ($\alpha = 0.5$)	1	0.614 (± 0.10)	0.878 (± 0.10)	0.389 (± 0.06)	0.785 (± 0.01)	0.607 (± 0.03)
	2	0.822 (± 0.08)	1.000 (–)	0.628 (± 0.05)	0.737 (± 0.01)	0.838 (± 0.02)
	4	0.918 (± 0.05)	1.000 (–)	0.790 (± 0.03)	0.641 (± 0.01)	0.935 (± 0.01)
TRIPBUILDER ($\alpha = 0.75$)	1	0.612 (± 0.10)	0.876 (± 0.10)	0.388 (± 0.06)	0.785 (± 0.01)	0.604 (± 0.03)
	2	0.820 (± 0.08)	1.000 (–)	0.628 (± 0.05)	0.737 (± 0.01)	0.837 (± 0.02)
	4	0.916 (± 0.05)	1.000 (–)	0.789 (± 0.03)	0.640 (± 0.01)	0.935 (± 0.01)
TRIPBUILDER ($\alpha = 1.0$)	1	0.609 (± 0.10)	0.872 (± 0.10)	0.391 (± 0.06)	0.785 (± 0.01)	0.602 (± 0.03)
	2	0.817 (± 0.08)	1.000 (–)	0.629 (± 0.05)	0.737 (± 0.01)	0.835 (± 0.02)
	4	0.917 (± 0.05)	1.000 (–)	0.791 (± 0.03)	0.641 (± 0.02)	0.935 (± 0.01)

this reason, we do not report its results in Fig. 6. Results confirm that both the approaches for TRAJSP can be exploited in practice. In general, Nearest Trajectory is always faster than Local Search. However, since the execution times are in any case limited, and Local Search remarkably outperforms Nearest Trajectory in effectiveness it has to be preferred.

6. Related work

Several works in the literature propose methods to recommend a list of Pols based on the actual location of the user (location-based services) (Huang & Bian, 2009; Monreale, Pinelli, Trasarti, & Giannotti, 2009; Yoon, Zheng, Xie, & Woo, 2010, 2012; Lucchese, Perego, Silvestri, Vahabi, & Venturini, 2012). However, in this paper we focus on a method for suggesting “sightseeing tours” based on time-budgeted itineraries matching the user preferences. Even if TRIPBUILDER could be easily adapted to fit a location-based service (LBS) model, we enhance classical LBS services by precomputing itineraries

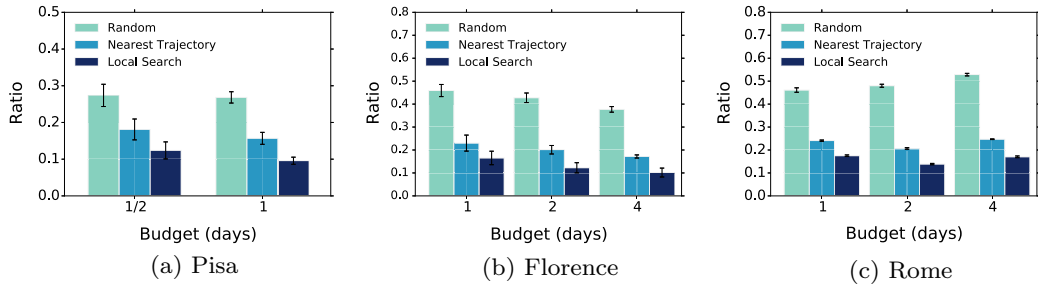


Fig. 4. Average path costs of the techniques employed to solve TrajSP as a function of the budget.

recommended to the user **before** the visit, thus at the planning stage. The task of designing a trip for a tourist approaching a new city has been investigated in the literature as we can see in the interesting survey on the topic presented in Vansteenwegen and Souffriau (2010).

An early work on recommending sightseeing itineraries is (Godart, 1999). Here authors use the Traveling Salesman Problem (TSP) as a core problem to plan trips. They automatically select Pols and lodging trying to minimizing transport and accommodation costs while maximizing the attractiveness of the accommodation and the activities. Although interesting, this model is very complex and computationally expensive, making it very difficult to use in practice.

Shang et al. propose and investigate a problem called *User Oriented Trajectory Search* (UOTS) for trip recommendation (Shang et al., 2012). Given a trajectory data set, the query input contains a set of intended places given by the tourist and a set of textual attributes describing the tourist preference. If a trajectory is connecting/close to the specified query locations, and the textual attributes of the trajectory are similar to the tourist preference, it will be recommended to the tourist. This type of queries can bring benefits to tourists in many popular applications such as trip planning and recommendation. However, this approach does not take into account the user's time budget in order to optimize her trip.

An interesting approach to the trip recommendation problem is the one proposed by Vansteenwegen et al., where authors define the Tourist Trip Design Problems (TTDP) (Souffriau, Vansteenwegen, Vertommen, Berghe, & Van Oudheusden, 2008; Vansteenwegen & Van Oudheusden, 2007). The orienteering problem, from the operational research literature, is a starting point for modeling the TTDP. They start with a set of possible locations with a score (indicating the user interest) and the objective is to maximize the total score of the visited locations, while keeping the total time (or distance) below the available time budget. They applied their algorithm, based on a guided local search, and a competitor, Dynamic Tour Guide, to a real data set of tourist points of interest with descriptions from the city of Ghent. Their approach turns out to be faster than the competitor and produces solutions of better quality. They also develop (Vansteenwegen, Souffriau, Berghe, & Oudheusden, 2011) a tourist expert system, called the "City Trip Planner" for planning routes.

The orienteering problem is also employed in De Choudhury et al. (2010). Here, De Choudhury et al. construct intra-city travel itineraries automatically by tapping a latent source reflecting geo-temporal traces left by millions of tourists. To do so, as in our case they firstly extract photo streams of individual users from Flickr. In the second step, they aggregate all user photo streams into a Pol graph. Itineraries are then automatically constructed from the graph based on the popularity of the Pols and subject to the user's time and destination constraints. However, the proposed orienteering problem does not model user preferences and it also does not model co-visitation of different Pols. Furthermore, the greedy algorithm solving the orienteering problem explicitly needs a source Pol, a destination Pol, the total number of Pols to be visited in the trip and a possible set of Pols to not be visited.

Lu, Lin, and Tseng (2011), propose Trip-Mine to efficiently find the optimal trip which satisfies the user's travel time constraint based on the user's location. The authors evaluate Trip-Mine in terms of mining efficiency and memory storage

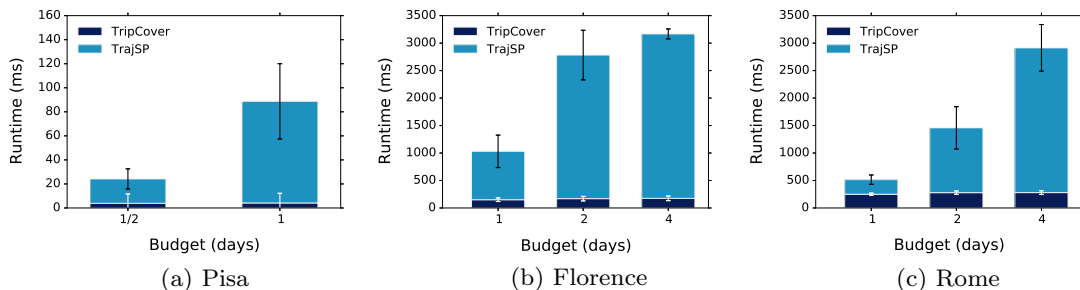


Fig. 5. Average total runtime as a function of the time budget for each city. Bottom bars refer to the average TRIPCOVER runtime while upper bars refer to the average TrajSP runtime.

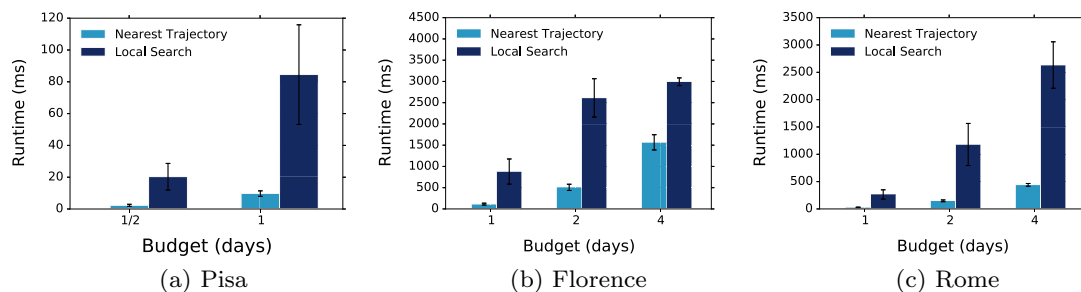


Fig. 6. Average runtime of Nearest Trajectory and Local Search by varying the time budget.

requirement for optimal trip finding. In Zheng and Xie (2011), Zheng et al. discuss the issue of mining interesting locations and classical travel sequences. In this work, they propose a HITS-based inference model to mine interesting locations and classical travel sequences based on multiple users GPS trajectories in a given geospatial region.

TRIPBUILDER overcomes some of the limitations of the works listed above. At the best of our knowledge it is the only system that solves the budgeted trip planning problem in a completely unsupervised way by exploiting user-generated content mined from Wikipedia and Flickr. Our principled modeling of the tour planning problem as a sequence of TRIPCOVER and TRAJSP instances along with the algorithms we proposed to address these problem allowed us to build an efficient and robust system to efficiently compute personalized itineraries that fits precisely the user preferences.

7. Conclusions

In this paper, we comprehensively discussed TRIPBUILDER, our unsupervised framework for recommending personalized sightseeing tours. TRIPBUILDER addresses the problem of planning the visit to a city of touristic interest as a two-steps process. First, given the profile of the user and the amount of time available for the visit, the set of itineraries across the Pols that best fits the user interests and respect the time constraint is chosen. This is done by solving the TRIPCOVER problem by means of an approximation algorithm. Then, the selected trajectories are joined in a sightseeing itinerary by means of a local search algorithm addressing TRAJSP, a particular instance of the Traveling Salesman Problem. TRIPBUILDER generates the budgeted sightseeing tours by composing popular trajectories performed by real tourists as mined from Flickr Photo albums. Moreover, since both Pols characteristics and user preferences are mapped into the same categorization automatically extracted from Wikipedia, it is able to personalize the recommended itinerary and even consider the popularity of each Pols as estimated from the number of photos available for it.

We experimented TRIPBUILDER with data collected for three cities different for their size and the amount of user-generated content available for download. The process exploited to mine such content from Flickr, Wikipedia and Google Maps and to build the TRIPBUILDER knowledge base has been detailed, and an analysis of the data collected has been provided. We evaluated our framework by considering the performance of the algorithms proposed to address both the TRIPCOVER and the TRAJSP problems. The proposed solutions resulted to outperforms the baselines in terms of all the metrics adopted for assessment. Our solution suggests itineraries that better match user preferences. Moreover, such itineraries present higher visiting time and, consequently, lower intra-Pol movement time than the baselines. Furthermore, we assessed the performance of our TSP-based local search heuristic to schedule a set of trajectories into the user agenda. Finally, the tests conducted to demonstrate the efficiency of TRIPBUILDER show that it computes a four-day personalized sightseeing tours of Rome in about 3 s thus confirming that our approach can be fruitfully deployed in online applications.

Acknowledgments

We acknowledge Liran Katzir and all authors of Cohen and Katzir (2008) for providing us their GMC source code. This work was partially supported by EU FP7 Marie Curie project SEEK (No. 295179), PRIN 2011 project ARS TECNOMEDIA, CNPQ Scholarship (No. 306806/2012-6), CNPQ Casadinho/PROCAD Project (No. 552578/2011-8), CNPQ-CNR Bilateral Project (No. 490459/2011-0), CNPQ Scholarship (No. 211836/2013-3), and by the Italian Ministry of Economic Development Project MOTUS (No. MS01 00015 – Industria2015).

References

- Baeza-Yates, R., Ribeiro-Neto, B., et al (1999). *Modern information retrieval* (Vol. 463). New York: ACM Press.
- Brilhante, I., Macedo, J. A., Nardini, F. M., Perego, R., & Renso, C. (2013). Where shall we go today?: Planning touristic tours with tripbuilder. In *Proceedings of the 22nd ACM international conference on information and knowledge management CIKM 2013* (pp. 757–762). New York, NY, USA: ACM.
- Brilhante, I., Macedo, J. A., Nardini, F. M., Perego, R., & Renso, C. (2014). Tripbuilder: A tool for recommending sightseeing tours. In M. Rijke, T. Kenter, A. P. Vries, C. X. Zhai, F. Jong, & K. Radinsky, et al. (Eds.), *Advances in information retrieval. Lecture notes in computer science* (Vol. 8416, pp. 771–774). Berlin: Springer International Publishing.

- Cohen, R., & Katzir, L. (2008). The generalized maximum coverage problem. *Information Processing Letters*, 108, 15–22.
- De Choudhury, M., Feldman, M., Amer-Yahia, S., Golbandi, N., Lempel, R., & Yu, C. (2010). Automatic construction of travel itineraries using social breadcrumbs. In *Proc. HT* (pp. 35–44). ACM.
- Ester, M., Peter Kriegl, H., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise (pp. 226–231). AAAI Press.
- Godart, J.-M. (1999). Combinatorial optimisation based decision support system for trip planning. In *Information and communication technologies in tourism 1999* (pp. 318–327). Springer.
- Huang, Y., & Bian, L. (2009). A bayesian network and analytic hierarchy process based personalized recommendations for tourist attractions over the internet. *Expert Systems with Applications*, 36, 933–943.
- Lucchese, C., Perego, R., Silvestri, F., Vahabi, H., & Venturini, R. (2012). How random walks can help tourism. In *Proc. ECIR*. LNCS.
- Lu, E., Lin, C., & Tseng, V. (2011). Trip-mine: An efficient trip planning approach with travel time constraints. *2011 12th IEEE International Conference on Mobile data management (MDM)* (Vol. 1, pp. 152–161). IEEE.
- Monreale, A., Pinelli, F., Trasarti, R., & Giannotti, F. (2009). Wherenext: A location predictor on trajectory pattern mining. In *Proc. KDD* (pp. 637–646). ACM.
- Rajesh Matai, S. S., & Mittal, M. L. (2010). Traveling salesman problem: An overview of applications, formulations, and solution approaches, traveling salesman problem, theory and applications. InTech. <http://dx.doi.org/10.5772/547>.
- Shang, S., Ding, R., Yuan, B., Xie, K., Zheng, K., & Kalnis, P. (2012). User oriented trajectory search for trip recommendation. In *Proc. EDBT* (pp. 156–167). ACM.
- Souffriau, W., Vansteenwegen, P., Vertommen, J., Berghe, G., & Van Oudheusden, D. (2008). A personalized tourist trip design algorithm for mobile tourist guides. *Applied Artificial Intelligence*, 22, 964–985.
- Vansteenwegen, P., & Souffriau, W. (2010). Trip planning functionalities: State of the art and future. *Information Technology & Tourism*, 12, 305–315.
- Vansteenwegen, P., Souffriau, W., Berghe, G., & Oudheusden, D. (2011). The city trip planner: An expert system for tourists. *Expert Systems with Applications*, 38, 6540–6546.
- Vansteenwegen, P., & Van Oudheusden, D. (2007). The mobile tourist guide: An or opportunity. *OR Insight*, 20, 21–27.
- Yoon, H., Zheng, Y., Xie, X., & Woo, W. (2010). Smart itinerary recommendation based on user-generated GPS trajectories. In *Ubiquitous intelligence and computing* (pp. 19–34).
- Yoon, H., Zheng, Y., Xie, X., & Woo, W. (2012). Social itinerary recommendation from user-generated digital trails. *Personal and Ubiquitous Computing*, 16, 469–484.
- Zheng, Y., & Xie, X. (2011). Learning travel recommendations from user-generated GPS traces. *ACM Transactions on Intelligent Systems and Technology*, 2.